

Министерство науки и высшего образования РФ

Федеральное государственное автономное образовательное учреждение
высшего образования «Казанский (Приволжский) федеральный университет»

**ИНСТИТУТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ
И ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ**

**КАФЕДРА ПРИКЛАДНОЙ МАТЕМАТИКИ
И ИСКУССТВЕННОГО ИНТЕЛЛЕКТА**

Направление: 01.03.04 – Прикладная математика

Профиль: Прикладная математика

ОТЧЁТ

по лабораторной работе

«Многослойный персептрон»

Выполнил: студент группы 09-322

Каримов А.И.

Проверил: _____

Казань, 2026

Содержание

1	Введение	2
2	Анализ и предобработка данных	2
3	Математическое обоснование	2
3.1	Архитектура сети	2
3.2	Функции активации	3
3.3	Функция потерь и обратное распространение	3
3.4	Оценка модели	3
4	Реализация ключевых алгоритмов	3
4.1	Прямое распространение (forward pass)	4
4.2	Обратное распространение (backward pass)	4
4.3	Обновление весов (градиентный спуск)	4
4.4	Функция потерь (бинарная кросс-энтропия)	5
5	Результаты	5
5.1	Кривые обучения	5
6	Заключение	6

1 Введение

Целью данного проекта является реализация алгоритма многослойного персептрона (MLP) с нуля на языке Python для решения задачи бинарной классификации. В ходе работы были применены знания линейной алгебры и математического анализа (вычисление производных сложной функции) для реализации прямого и обратного распространения ошибки, а также градиентного спуска.

2 Анализ и предобработка данных

Набор данных `data.csv` содержит 32 столбца:

- `id`: уникальный идентификатор образца (удален, так как не несет семантической нагрузки).
- `diagnosis`: целевая переменная (М – злокачественная, В – доброкачественная).
- 30 числовых признаков, описывающих характеристики ядра клетки (радиус, текстура, периметр, площадь, гладкость и т.д.)

Предобработка:

1. Кодирование меток: $V \rightarrow 0$, $M \rightarrow 1$.
2. Стандартизация признаков (Z-score normalization):

$$x' = \frac{x - \mu}{\sigma} \quad (1)$$

где μ – среднее, σ – стандартное отклонение. Это необходимо для стабильной работы градиентного спуска.

3. Разделение данных: 80% на обучение, 20% на проверку с фиксированным `seed=42` для воспроизводимости.

3 Математическое обоснование

3.1 Архитектура сети

Модель содержит два скрытых слоя. Для слоя l прямой проход вычисляется как:

$$Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]} \quad (2)$$

где $A^{[0]} = X$ (входные данные).

3.2 Функции активации

Для скрытых слоев используется функция **ReLU**:

$$\text{ReLU}(z) = \max(0, z), \quad \frac{\partial \text{ReLU}}{\partial z} = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{if } z \leq 0 \end{cases} \quad (3)$$

На выходном слое используется функция **Softmax** для получения вероятностного распределения по двум классам:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (4)$$

3.3 Функция потерь и обратное распространение

Для обучения используется категориальная кросс-энтропия (совместимая с Softmax):

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^2 y_{ik} \log(\hat{y}_{ik}) \quad (5)$$

Производная функции потерь по выходу слоя Softmax имеет элегантный вид: $\frac{\partial \mathcal{L}}{\partial Z^{[L]}} = \hat{Y} - Y$.

Далее ошибка распространяется назад по сети:

$$dW^{[l]} = \frac{1}{m} (A^{[l-1]})^T dZ^{[l]} \quad (6)$$

$$db^{[l]} = \frac{1}{m} \sum dZ^{[l]} \quad (7)$$

$$dZ^{[l]} = dZ^{[l+1]} (W^{[l+1]})^T \odot \text{ReLU}'(Z^{[l]}) \quad (8)$$

Обновление весов осуществляется с помощью градиентного спуска: $W^{[l]} = W^{[l]} - \alpha dW^{[l]}$.

3.4 Оценка модели

Согласно заданию, итоговая оценка производится с помощью **бинарной кросс-энтропии** (BCE) для вероятности положительного класса (М):

$$\text{BCE} = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (9)$$

где $\hat{p}_i$ – вероятность класса М, предсказанная сетью.

4 Реализация ключевых алгоритмов

Ниже приведены наиболее важные фрагменты кода, иллюстрирующие работу прямого и обратного распространения ошибки, а также функции активации.

4.1 Прямое распространение (forward pass)

На каждом слое вычисляется линейная комбинация входов с весами, после чего применяется функция активации. Кэш сохраняется для последующего использования в обратном проходе.

```
1     def forward_layer(self, A_prev, W, b, activation):
2         Z = np.dot(A_prev, W) + b
3         self.cache['Z'].append(Z)
4         if activation == 'relu':
5             A = np.maximum(0, Z)                # ReLU
6         elif activation == 'softmax':
7             exp_Z = np.exp(Z - np.max(Z, axis=1, keepdims=True))
8             A = exp_Z / np.sum(exp_Z, axis=1, keepdims=True)
9         self.cache['A'].append(A)
10        return A
```

Листинг 1: Прямое распространение по одному слою

Пояснение. Вычитание максимума в числителе softmax — стандартный приём для борьбы с переполнением экспоненты (exp overflow), не влияющий на итоговое распределение.

4.2 Обратное распространение (backward pass)

Градиенты вычисляются рекурсивно, начиная с выходного слоя. Для комбинации Softmax + Categorical Cross-Entropy производная упрощается до разности предсказания и истинной метки.

```
1     def backward(self, Y_onehot):
2         m = Y_onehot.shape[0]
3         L = len(self.layers) - 1
4         dZ = self.cache['A'][-1] - Y_onehot
5         for l in reversed(range(1, L + 1)):
6             A_prev = self.cache['A'][l - 1]
7             dW = np.dot(A_prev.T, dZ) / m
8             db = np.sum(dZ, axis=0, keepdims=True) / m
9             if l > 1:
10                dZ = np.dot(dZ, self.weights[l].T) * (self.cache['Z'][l-1] > 0)
11            self.grads[f'dW{l}'] = dW
12            self.grads[f'db{l}'] = db
```

Листинг 2: Обратное распространение ошибки

4.3 Обновление весов (градиентный спуск)

```
1 def update_weights(self, lr):
2     for l in range(1, len(self.layers)):
3         self.weights[l] -= lr * self.grads[f'dW{l}']
4         self.biases[l] -= lr * self.grads[f'db{l}']
```

Листинг 3: Шаг градиентного спуска

4.4 Функция потерь (бинарная кросс-энтропия)

Для итоговой оценки модели используется BCE по вероятности положительного класса:

```
1 def binary_cross_entropy(y_true, y_pred):
2     eps = 1e-12
3     y_pred = np.clip(y_pred, eps, 1 - eps)
4     return -np.mean(
5         y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred)
6     )
```

Листинг 4: Вычисление бинарной кросс-энтропии

Пояснение. Ограничение `clip` предотвращает вычисление $\log(0)$, что могло бы привести к NaN в значении функции потерь.

5 Результаты

При запуске обучения с архитектурой [30, 16, 8, 2], скоростью обучения $\alpha = 0.1$ и 300 эпохами, модель демонстрирует быструю сходимость.

- Точность на проверочной выборке (Val Accuracy) достигает $\approx 97\text{--}98\%$.
- Значение бинарной кросс-энтропии (BCE) стабилизируется на уровне 0.08–0.12, что указывает на высокую уверенность модели в правильных предсказаниях.
- Графики кривых обучения не показывают признаков переобучения (train и val loss уменьшаются синхронно).

5.1 Кривые обучения

На рисунке представлены зависимости функции потерь и точности от номера эпохи для обучающей и проверочной выборок.

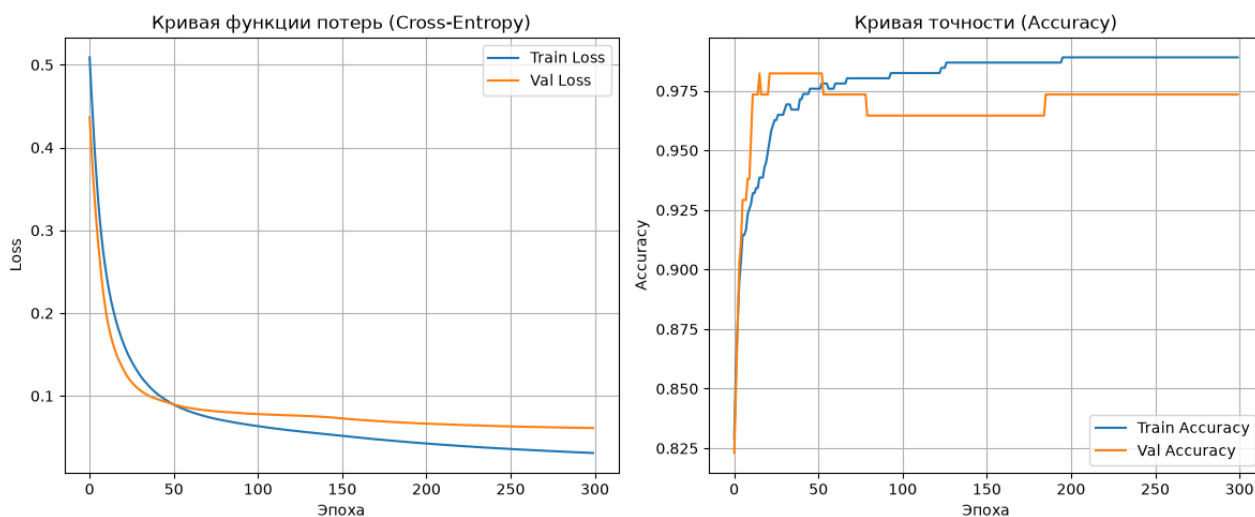


Рис. 1: Кривые обучения: функция потерь (Cross-Entropy) и точность (Accuracy) на обучающей и проверочной выборках

Как видно из рисунка, кривые обучения и проверки сходятся без существенного расхождения, что свидетельствует об отсутствии переобучения при выбранной архитектуре и скорости обучения.

6 Заключение

В ходе выполнения проекта была успешно реализована архитектура многослойного персептрона без использования специализированных библиотек глубокого обучения. Были закреплены навыки работы с матричными операциями в NumPy, вычисления градиентов методом обратного распространения ошибки, а также проанализированы метрики качества классификации. Построенные кривые обучения подтверждают корректность реализации и стабильность процесса оптимизации.